

Deterministic PyTorch regression tests

Rohit Goswami · 2026-03-11 · pytorch · testing · python

PyTorch has a surprising amount of non-determinism baked into operations that look like they should be exact. `index_add_` in particular accumulates floating-point values across threads, and the summation order changes between runs depending on how the OS schedules those threads. The result: regression tests that pass locally, fail in CI, pass again on retry, and generally make everyone question their sanity.

The fix requires clamping the thread count before each test:

```
@pytest.fixture(autouse=True)
def deterministic_threads():
    n_threads = torch.get_num_threads()
    torch.set_num_threads(1)
    yield
    torch.set_num_threads(n_threads)
```

`torch.set_num_threads(1)` forces single-threaded accumulation, so `index_add_` (and friends) always sum in the same order. The fixture restores the original thread count afterward to avoid poisoning other tests.

`DataLoader` has a separate source of divergence: when `num_workers > 0`, each worker subprocess gets its own RNG state. Even with a fixed seed in the main process, the workers can diverge across runs. Setting `num_workers=0` eliminates the issue by doing all loading in the main process.

With both of these in place, results become deterministic on a given machine. But “given machine” is doing a lot of work in that sentence. Training amplifies cross-hardware float differences (different SIMD paths, math libraries) through backprop and optimizer steps, so CI runners can diverge from local machines by up to ~ 0.02 absolute. The tolerances end up reflecting that:

```
torch.testing.assert_close(result, expected, rtol=5e-3, atol=0.05)
```

Still a large improvement over the pre-thread-pinning situation, where non-deterministic accumulation on a single machine forced even wider tolerances and made regression tests effectively meaningless. The remaining gap comes from legitimate hardware differences, not from run-to-run randomness.

See [metatensor/metatrain#1070](https://github.com/pytorch/pytorch/pull/1070) for the full changeset.